

Proyecto de Grado: IA al servicio de las comunidades

Maestría en Analítica para la Inteligencia de Negocios

Nombres: María Fernanda Izquierdo Aparicio & Ana María Ochoa Muñoz **Fecha:** 17 de Marzo de 2025

Este modelo forma parte del proyecto de grado de la Maestría en Analítica para la Inteligencia de Negocios de la Pontificia Universidad Javeriana, enfocado en la aplicación de Inteligencia Artificial al servicio de las comunidades. Su objetivo es clasificar imágenes de plantas en función de su especie.

1. Importe de librerías

Se cargan las librerías necesarias para manipulación de datos, procesamiento de imágenes y modelado con redes neuronales. Algunas destacadas:

- Pandas y NumPy: Para manejo de datos.
- Matplotlib y Seaborn: Para visualización.
- OpenCV (cv2) y PIL (Image): Para procesamiento de imágenes.
- TensorFlow y Keras: Para construir y entrenar modelos de deep learning.

```
In [112... import json

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as mpimg

import os
import zipfile
import shutil
import seaborn as sns
import datetime
import cv2

import keras
from keras import models
from keras import layers
```

```

import random

import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Layer
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.preprocessing import image
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.preprocessing.image import img_to_array, load_img
from tensorflow.keras.callbacks import ReduceLROnPlateau
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping, ReduceLROnPlateau
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras.optimizers import Adam

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score

from PIL import Image # Librería para abrir y manipular imágenes

# ignoring warnings
import warnings
warnings.simplefilter("ignore")

```

```

In [4]: #Medir_tiempo_ejecucion():
        inicio = datetime.datetime.now()
        print(f"Inicio: {inicio.strftime('%Y-%m-%d %H:%M:%S.%f')}")

```

Inicio: 2025-04-04 13:35:02.092313

2. Importe de Datos

- Se importa un archivo CSV (Data_Final.csv) con el listado de imágenes y etiquetas.
- Se eliminan registros de la clase "Orange".
- Se divide el dataset en 80% entrenamiento - 20% prueba, asegurando balance entre clases con stratify.

```

In [6]: train=pd.read_csv('Data_Final.csv', sep=";")
        train.head()

```

Out [6]:

	Carpeta	Nombre del archivo	Planta	Condición	Saludable
0	Strawberry___healthy	8f558908-aa1b-4a86-855a-5094c2392e5a___RS_HL1...	Strawberry	healthy	SI Stra
1	Strawberry___healthy	b8e9ed27-8e37-4214-9206-f8c0ef21cf4d___RS_HL4...	Strawberry	healthy	SI Stra
2	Strawberry___healthy	abdd34a0-ab02-41e0-95a3-a014ab863ec2___RS_HL1...	Strawberry	healthy	SI Stra
3	Strawberry___healthy	d1aee44a-b6bb-45b9-b7b6-5d553add8fd1___RS_HL2...	Strawberry	healthy	SI Stra
4	Strawberry___healthy	3d28c3ea-8419-4e09-addd-211e3828e39f___RS_HL1...	Strawberry	healthy	SI Stra

In [7]:

```
#Quitar las naranjas
train = train[train["Planta"] != "Orange"]
```

3. Análisis de Datos y Creación de conjuntos de train & test

- Se analiza la distribución de imágenes por categoría mediante gráficos de conteo.

Se visualizan imágenes de plantas saludables y enfermas usando OpenCV.

Se obtiene el tamaño de las imágenes para estandarizar el preprocesamiento.

3.1. Conteo de imágenes por categoría Plantas (Especies)

In [10]:

```
train.Planta.value_counts()
```

```
Out[10]: Planta
Yuca          21397
Tomato        18160
Grape         4062
Corn_(maize)  3852
Apple         3174
Pepper_bell   2475
Peach         2297
Potato        2152
Strawberry    1565
Name: count, dtype: int64
```

3.2. Division aleatoria entre conjunto de train y test usando una division 80% - 20%

```
In [12]: train, test = train_test_split( train, test_size=0.2, random_state=42, shuffle=True,
train.shape, test.shape
```

```
Out[12]: ((47307, 6), (11827, 6))
```

3.3. Conteo de imágenes por en el conjunto train por categoría

```
In [14]: train.Planta.value_counts()
```

```
Out[14]: Planta
Yuca          17117
Tomato        14528
Grape         3249
Corn_(maize)  3082
Apple         2539
Pepper_bell   1980
Peach         1838
Potato        1722
Strawberry    1252
Name: count, dtype: int64
```

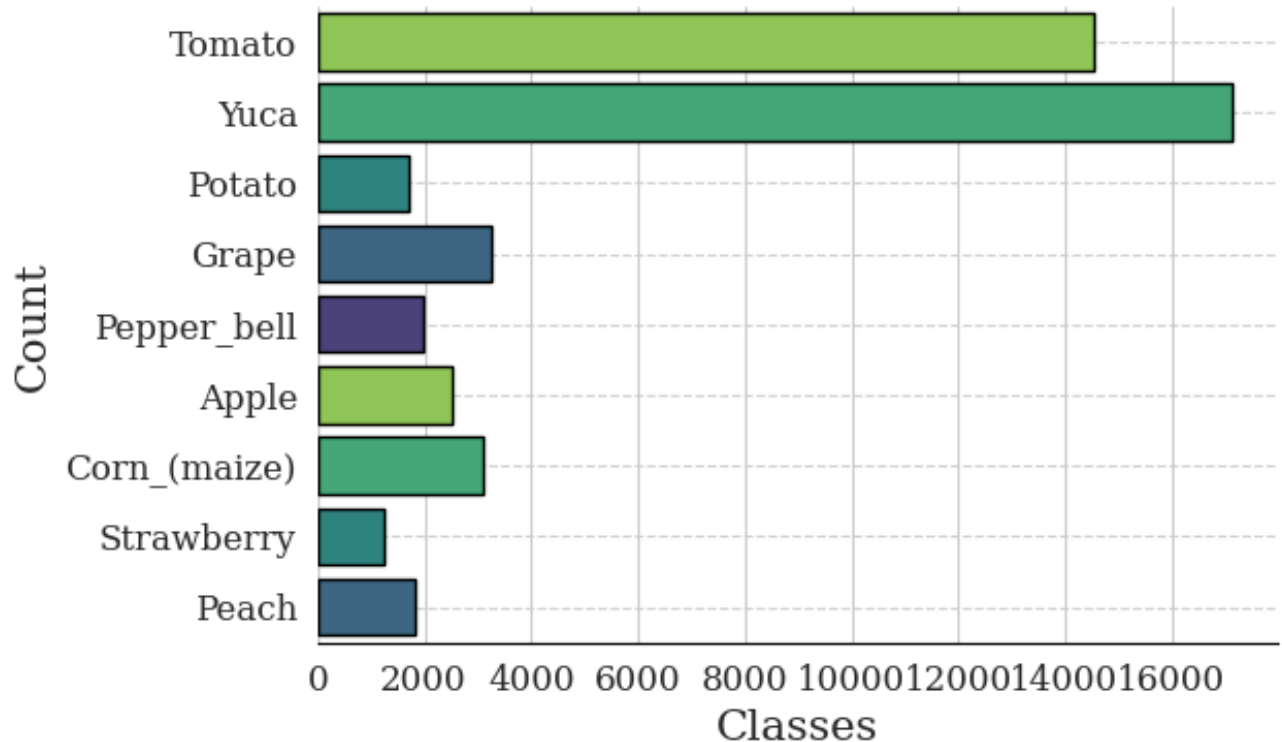
```
In [15]: sns.set_style("whitegrid")
fig, ax = plt.subplots(figsize = (6, 4))

for i in ['top', 'right', 'left']:
    ax.spines[i].set_visible(False)
ax.spines['bottom'].set_color('black')

sns.countplot(train.Planta, edgecolor = 'black',
               palette = reversed(sns.color_palette("viridis", 5)))
plt.xlabel('Classes', fontfamily = 'serif', size = 15)
plt.ylabel('Count', fontfamily = 'serif', size = 15)
```



```
plt.xticks(fontfamily = 'serif', size = 12)
plt.yticks(fontfamily = 'serif', size = 12)
ax.grid(axis = 'y', linestyle = '--', alpha = 0.9)
plt.show()
```



3.4. Conteo de imágenes por en el conjunto test por categoría

```
In [17]: test.Planta.value_counts()
```

```
Out[17]: Planta
Yuca          4280
Tomato        3632
Grape         813
Corn_(maize)  770
Apple         635
Pepper_bell   495
Peach         459
Potato        430
Strawberry    313
Name: count, dtype: int64
```

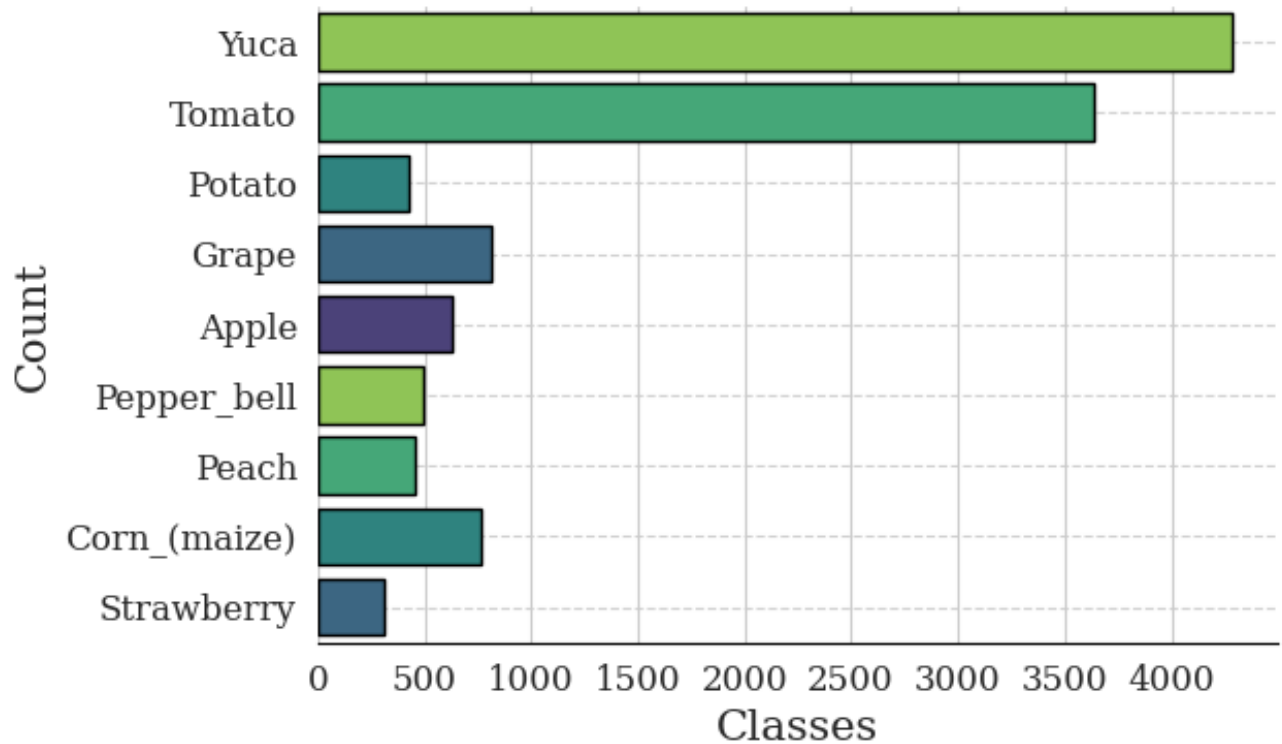
```
In [18]: sns.set_style("whitegrid")
fig, ax = plt.subplots(figsize = (6, 4))

for i in ['top', 'right', 'left']:
    ax.spines[i].set_visible(False)
ax.spines['bottom'].set_color('black')
```

```

sns.countplot(test.Planta, edgecolor = 'black',
               palette = reversed(sns.color_palette("viridis", 5)))
plt.xlabel('Classes', fontfamily = 'serif', size = 15)
plt.ylabel('Count', fontfamily = 'serif', size = 15)
plt.xticks(fontfamily = 'serif', size = 12)
plt.yticks(fontfamily = 'serif', size = 12)
ax.grid(axis = 'y', linestyle = '--', alpha = 0.9)
plt.show()

```



3.5. Muestra de Imagenes

```

In [20]: # Cambiar el nombre de la columna
test = test.rename(columns={'Nombre del archivo': 'image_id'})
train = train.rename(columns={'Nombre del archivo': 'image_id'})

```

```

In [21]: train.head()

```

Out [21]:

	Carpeta	image_id	Planta	Conc
38289	Tomato___Septoria_leaf_spot	53f8a06e-20f1-4433-b558-01f7e5d14dba___Matt.S...	Tomato	Septoria_leaf
60108	Casava	4034332345.jpg	Yuca	Cassava M Disease (
33217	Tomato___Spider_mites Two-spotted_spider_mite	3aad576c-3189-4b78-a5df-c9bac0078612___Com.G_S...	Tomato	Spider_mites spotted_spider
6146	Potato___Late_blight	9570095c-02e9-497b-bec1-4e526f3462e5___RS_LB 2...	Potato	Late_
63587	Casava	795796251.jpg	Yuca	Cassava (Mottle (

Plantas Healthy

```
In [23]: # Seleccionar 3 muestras de la clase "Healthy"
sample = train[train.Saludable == "SI"].sample(3)

# Tamaño de la figura
plt.figure(figsize=(15, 5))

# Carpeta donde están las imágenes
image_folder = "Datos Finales"

for ind, (image_id, Planta) in enumerate(zip(sample.image_id, sample.Planta)):
    plt.subplot(1, 3, ind + 1)

    # Construir la ruta completa de la imagen
    image_path = os.path.join(image_folder, image_id)

    # Cargar la imagen correctamente
    img = cv2.imread(image_path)

    # Verificar si la imagen se cargó correctamente
    if img is None:
        print(f"Error al cargar la imagen: {image_path}")
        continue

    # Convertir la imagen de BGR a RGB
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    # Mostrar la imagen
    plt.imshow(img)
    plt.axis("off")
```

```
plt.show()
```



Plantas con alguna enfermedad

```
In [25]: # Seleccionar 3 muestras de la clase "Healthy"
sample = train[train.Saludable == "NO"].sample(3)

# Tamaño de la figura
plt.figure(figsize=(15, 5))

# Carpeta donde están las imágenes
image_folder = "Datos Finales"

for ind, (image_id, Planta) in enumerate(zip(sample.image_id, sample.Planta)):
    plt.subplot(1, 3, ind + 1)

    # Construir la ruta completa de la imagen
    image_path = os.path.join(image_folder, image_id)

    # Cargar la imagen correctamente
    img = cv2.imread(image_path)

    # Verificar si la imagen se cargó correctamente
    if img is None:
        print(f"Error al cargar la imagen: {image_path}")
        continue

    # Convertir la imagen de BGR a RGB
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    # Mostrar la imagen
    plt.imshow(img)
    plt.axis("off")

plt.show()
```



Obtener el tamaño de las imágenes

```
In [27]: image_path = os.path.join(image_folder, image_id)

# Definir la carpeta con las imágenes
image_folder = "Datos Finales"

# Función para obtener el tamaño de una imagen
def get_image_size(image_path):
    try:
        with Image.open(image_path) as img:
            return img.size # Devuelve (ancho, alto)
    except Exception as e:
        print(f"Error con la imagen {image_path}: {e}")
        return None, None
```

```
In [28]: # Obtener el tamaño de cada imagen y agregarlo al DataFrame
widths = []
heights = []

for img_name in train["image_id"]: # Asegúrate de que la columna se llama "
    img_path = os.path.join(image_folder, img_name)
    width, height = get_image_size(img_path)
    widths.append(width)
    heights.append(height)

train["width"] = widths
train["height"] = heights

# Mostrar las primeras filas con los nuevos datos
print(train.head())
```

```
Error con la imagen Datos Finales/db19f539-05e8-4cd8-a9bc-57fb97ce0f41__RS_
NLB 4443 copy.jpg: [Errno 2] No such file or directory: '/Users/mfia/Docume
nts/MFIA - PUJ - Maestría/Proyecto de Grado/0. Data/Datos Finales/db19f539-0
5e8-4cd8-a9bc-57fb97ce0f41__RS_NLB\xa0 4443 copy.jpg'
Error con la imagen Datos Finales/90dd4436-0614-4f6b-ad3b-b8db56ce4de8__RS_
NLB 4673.JPG: [Errno 2] No such file or directory: '/Users/mfia/Documents/M
```

FIA - PUJ - Maestría/Proyecto de Grado/0. Data/Datos Finales/90dd4436-0614-4f6b-ad3b-b8db56ce4de8__RS_NLB\xa0 4673.JPG'

Error con la imagen Datos Finales/archivos_plantvillage.csv: cannot identify image file '/Users/mfia/Documents/MFIA - PUJ - Maestría/Proyecto de Grado/0. Data/Datos Finales/archivos_plantvillage.csv'

Error con la imagen Datos Finales/ea9ba5b2-e8e7-48c8-ac16-d594a9e5cc51__RS_NLB 4443 copy 2.jpg: [Errno 2] No such file or directory: '/Users/mfia/Documents/MFIA - PUJ - Maestría/Proyecto de Grado/0. Data/Datos Finales/ea9ba5b2-e8e7-48c8-ac16-d594a9e5cc51__RS_NLB\xa0 4443 copy 2.jpg'

Error con la imagen Datos Finales/62be90ae-831c-4e09-af81-8e73ab7f56c7__RS_NLB 4442.JPG: [Errno 2] No such file or directory: '/Users/mfia/Documents/MFIA - PUJ - Maestría/Proyecto de Grado/0. Data/Datos Finales/62be90ae-831c-4e09-af81-8e73ab7f56c7__RS_NLB\xa0 4442.JPG'

Error con la imagen Datos Finales/461baed2-cdf3-4ce0-9812-65f34484bb35__R.S_HL 8031 copy 3.jpg: [Errno 2] No such file or directory: '/Users/mfia/Documents/MFIA - PUJ - Maestría/Proyecto de Grado/0. Data/Datos Finales/461baed2-cdf3-4ce0-9812-65f34484bb35__R.S_HL 8031 copy\xa0 3.jpg'

Error con la imagen Datos Finales/571927a9-3b0a-4c5e-be0b-a3c9859d66fa__RS_NLB 4443 copy 3.jpg: [Errno 2] No such file or directory: '/Users/mfia/Documents/MFIA - PUJ - Maestría/Proyecto de Grado/0. Data/Datos Finales/571927a9-3b0a-4c5e-be0b-a3c9859d66fa__RS_NLB\xa0 4443 copy 3.jpg'

Error con la imagen Datos Finales/76ffac09-0eab-4e97-9500-3d8237910165__RS_NLB 4627.JPG: [Errno 2] No such file or directory: '/Users/mfia/Documents/MFIA - PUJ - Maestría/Proyecto de Grado/0. Data/Datos Finales/76ffac09-0eab-4e97-9500-3d8237910165__RS_NLB\xa0 4627.JPG'

Error con la imagen Datos Finales/201bc21a-5372-4c91-a263-ea36ed96665a__RS_NLB 4628.JPG: [Errno 2] No such file or directory: '/Users/mfia/Documents/MFIA - PUJ - Maestría/Proyecto de Grado/0. Data/Datos Finales/201bc21a-5372-4c91-a263-ea36ed96665a__RS_NLB\xa0 4628.JPG'

Error con la imagen Datos Finales/d46fe184-72f6-4b05-8376-b6e17edc5458__RS_NLB 4671.JPG: [Errno 2] No such file or directory: '/Users/mfia/Documents/MFIA - PUJ - Maestría/Proyecto de Grado/0. Data/Datos Finales/d46fe184-72f6-4b05-8376-b6e17edc5458__RS_NLB\xa0 4671.JPG'

Error con la imagen Datos Finales/3fdd99f1-a3df-4498-8b6a-b844621a72ca__RS_NLB 4670 copy.jpg: [Errno 2] No such file or directory: '/Users/mfia/Documents/MFIA - PUJ - Maestría/Proyecto de Grado/0. Data/Datos Finales/3fdd99f1-a3df-4498-8b6a-b844621a72ca__RS_NLB\xa0 4670 copy.jpg'

Error con la imagen Datos Finales/ed02da2f-8b98-49b1-a65d-ff7e47d61331__RS_NLB 4443.JPG: [Errno 2] No such file or directory: '/Users/mfia/Documents/MFIA - PUJ - Maestría/Proyecto de Grado/0. Data/Datos Finales/ed02da2f-8b98-49b1-a65d-ff7e47d61331__RS_NLB\xa0 4443.JPG'

Error con la imagen Datos Finales/bd30a64c-92a4-422c-9bc1-7752f09e2809__RS_NLB 4507 copy.jpg: [Errno 2] No such file or directory: '/Users/mfia/Documents/MFIA - PUJ - Maestría/Proyecto de Grado/0. Data/Datos Finales/bd30a64c-92a4-422c-9bc1-7752f09e2809__RS_NLB\xa0 4507 copy.jpg'

Error con la imagen Datos Finales/6f370eac-7a90-40a3-be93-62147f32041c__RS_NLB 4608.JPG: [Errno 2] No such file or directory: '/Users/mfia/Documents/MFIA - PUJ - Maestría/Proyecto de Grado/0. Data/Datos Finales/6f370eac-7a90-40a3-be93-62147f32041c__RS_NLB\xa0 4608.JPG'

Error con la imagen Datos Finales/Untitled.ipynb: cannot identify image file

```

'/Users/mfia/Documents/MFIA - PUJ - Maestría/Proyecto de Grado/0. Data/Datos
Finales/Untitled.ipynb'
Error con la imagen Datos Finales/3fb57074-5e53-45e8-9219-bc0cee74f440__RS_
NLB_4597.JPG: [Errno 2] No such file or directory: '/Users/mfia/Documents/M
FIA - PUJ - Maestría/Proyecto de Grado/0. Data/Datos Finales/3fb57074-5e53-4
5e8-9219-bc0cee74f440__RS_NLB\xa0 4597.JPG'
Error con la imagen Datos Finales/8b2c9cfc-a522-4675-8fb0-87c627849b83__RS_
NLB_4672.JPG: [Errno 2] No such file or directory: '/Users/mfia/Documents/M
FIA - PUJ - Maestría/Proyecto de Grado/0. Data/Datos Finales/8b2c9cfc-a522-4
675-8fb0-87c627849b83__RS_NLB\xa0 4672.JPG'
Error con la imagen Datos Finales/.ipynb_checkpoints: [Errno 2] No such file
or directory: '/Users/mfia/Documents/MFIA - PUJ - Maestría/Proyecto de Grad
o/0. Data/Datos Finales/.ipynb_checkpoints'
Error con la imagen Datos Finales/fc1c72a4-1aff-4ebf-83ed-909a7b9dc382__RS_
NLB_4670.JPG: [Errno 2] No such file or directory: '/Users/mfia/Documents/M
FIA - PUJ - Maestría/Proyecto de Grado/0. Data/Datos Finales/fc1c72a4-1aff-4
ebf-83ed-909a7b9dc382__RS_NLB\xa0 4670.JPG'
Error con la imagen Datos Finales/f0b49686-1a0f-4c01-9b60-e816c513215f__RS_
NLB_4506_copy.jpg: [Errno 2] No such file or directory: '/Users/mfia/Docume
nts/MFIA - PUJ - Maestría/Proyecto de Grado/0. Data/Datos Finales/f0b49686-1
a0f-4c01-9b60-e816c513215f__RS_NLB\xa0 4506_copy.jpg'
Error con la imagen Datos Finales/ca1ab144-fb79-45cd-9cd4-c43408a26b49__RS_
NLB_4626.JPG: [Errno 2] No such file or directory: '/Users/mfia/Documents/M
FIA - PUJ - Maestría/Proyecto de Grado/0. Data/Datos Finales/ca1ab144-fb79-4
5cd-9cd4-c43408a26b49__RS_NLB\xa0 4626.JPG'

```

```

Carpeta \
38289          Tomato__Septoria_leaf_spot
60108          Casava
33217  Tomato__Spider_mites Two-spotted_spider_mite
6146          Potato__Late_blight
63587          Casava

image_id  Planta \
38289  53f8a06e-20f1-4433-b558-01f7e5d14dba__Matt.S...  Tomato
60108          4034332345.jpg  Yuca
33217  3aad576c-3189-4b78-a5df-c9bac0078612__Com.G_S...  Tomato
6146  9570095c-02e9-497b-bec1-4e526f3462e5__RS_LB 2...  Potato
63587          795796251.jpg  Yuca

```

```

Condición Saludable Planta_Estado  width
\
38289          Septoria_leaf_spot  NO  Tomato_sick  256.0
60108          Cassava Mosaic Disease (CMD)  NO  Yuca_sick  800.0
33217  Spider_mites Two-spotted_spider_mite  NO  Tomato_sick  256.0
6146          Late_blight  NO  Potato_sick  256.0
63587          Cassava Green Mottle (CGM)  NO  Yuca_sick  800.0

height
38289  256.0
60108  600.0

```

```
33217    256.0
6146     256.0
63587    600.0
```

4. Preparación del Modelo

- Se implementará data augmentation para mejorar la generalización.
- Se evaluará el modelo con métricas como precisión y recall.

```
In [30]: # Hiperparámetros principales
BATCH_SIZE = 8
STEPS_PER_EPOCH = len(train)*0.8 / BATCH_SIZE
VALIDATION_STEPS = len(train)*0.2 / BATCH_SIZE
EPOCHS = 8
TARGET_SIZE = 150
```

```
In [31]: train.Planta = train.Planta.astype('str')

train_datagen = ImageDataGenerator(validation_split = 0.2,
                                   preprocessing_function = None,
                                   rotation_range = 45,
                                   zoom_range = 0.2,
                                   horizontal_flip = True,
                                   vertical_flip = True,
                                   fill_mode = 'nearest',
                                   shear_range = 0.1,
                                   height_shift_range = 0.1,
                                   width_shift_range = 0.1)

train_generator = train_datagen.flow_from_dataframe(train,
                                                    directory = "Datos Finales",
                                                    subset = "training",
                                                    x_col = "image_id",
                                                    y_col = "Planta",
                                                    target_size = (TARGET_SIZE, TARGET_SIZE),
                                                    batch_size = BATCH_SIZE,
                                                    class_mode = "categorical")

validation_datagen = ImageDataGenerator(validation_split = 0.2)

validation_generator = validation_datagen.flow_from_dataframe(train,
                                                             directory = "Datos Finales",
                                                             subset = "validation",
                                                             x_col = "image_id",
                                                             y_col = "Planta",
                                                             target_size = (TARGET_SIZE, TARGET_SIZE),
                                                             batch_size = BATCH_SIZE,
```



```
class_mode = "categorical")
```

Found 37829 validated image filenames belonging to 9 classes.
Found 9457 validated image filenames belonging to 9 classes.

Imágen Aleatoria antes de Augmentation

```
In [33]: img_path = os.path.join("Datos Finales", train.image_id[20])
img = image.load_img(img_path, target_size = (TARGET_SIZE, TARGET_SIZE))
img_tensor = image.img_to_array(img)
img_tensor = np.expand_dims(img_tensor, axis = 0)
img_tensor /= 255.

plt.imshow(img_tensor[0])
plt.axis('off')
plt.show()
```

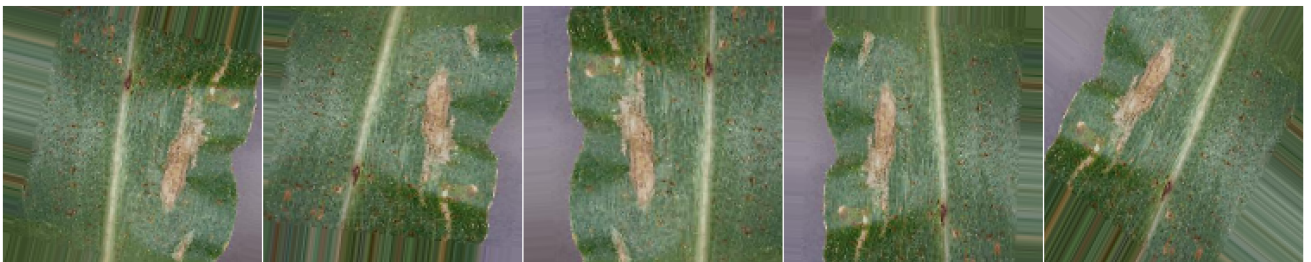


```
In [34]: generator = train_datagen.flow_from_dataframe(train.iloc[20:21],
    directory = "Datos Finales",
    x_col = "image_id",
    y_col = "Planta",
    target_size = (TARGET_SIZE, TARGET_SIZE),
    batch_size = BATCH_SIZE,
    class_mode = "categorical")

aug_images = [generator[0][0][0]/255 for i in range(10)]
```

```
fig, axes = plt.subplots(2, 5, figsize = (TARGET_SIZE, TARGET_SIZE))
axes = axes.flatten()
for img, ax in zip(aug_images, axes):
    ax.imshow(img)
    ax.axis('off')
plt.tight_layout()
plt.show()
```

Found 1 validated image filenames belonging to 1 classes.



5. Construcción del Modelo

5.1. Clasificador

Arquitectura del Modelo CNN El modelo es una red neuronal convolucional (**CNN**) diseñada para clasificar imágenes de plantas en distintas categorías. Su arquitectura incluye:

1. Capas Convolucionales y de MaxPooling Cada capa convolucional extrae características de las imágenes, mientras que **MaxPooling** reduce la dimensionalidad, mejorando la eficiencia del modelo.

- **Capa 1:**

- `Conv2D(32, (3,3), activation='relu', input_shape=`

- (150,150,3))
 - BatchNormalization()
 - MaxPooling2D(2,2)
- **Capa 2:**
 - Conv2D(64, (3,3), activation='relu')
 - BatchNormalization()
 - MaxPooling2D(2,2)
- **Capa 3:**
 - Conv2D(128, (3,3), activation='relu')
 - BatchNormalization()
 - MaxPooling2D(2,2)
- **Capa 4:**
 - Conv2D(128, (3,3), activation='relu')
 - BatchNormalization()
 - MaxPooling2D(2,2)

2. Capa de Aplanamiento Después de las capas convolucionales, se usa `Flatten()` para convertir la salida en un vector de una dimensión.

3. Capas Densas y Regularización

- Una capa densa con **512 neuronas** y función de activación ReLU.
- Se usa **Dropout (0.5)** para reducir el sobreajuste.

4. Capa de Salida

- `Dense(9, activation='softmax')`
- La activación **softmax** se usa porque se trata de un problema de clasificación multiclase (9 categorías).

```
In [37]: # Arquitectura del modelo CNN
model = models.Sequential()

# Capa 1: Convolucional y MaxPooling
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(150, 150, 3)))
model.add(layers.BatchNormalization())
model.add(layers.MaxPooling2D(2, 2))

# Capa 2: Convolucional y MaxPooling
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.BatchNormalization())
```

```
model.add(layers.MaxPooling2D(2, 2))

# Capa 3: Convolutional y MaxPooling
model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.BatchNormalization())
model.add(layers.MaxPooling2D(2, 2))

# Capa 4: Convolutional y MaxPooling
model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.BatchNormalization())
model.add(layers.MaxPooling2D(2, 2))

# Aplanamiento de la capa convolutional
model.add(layers.Flatten())

# Clasificador
model.add(layers.Dense(512, activation='relu'))
model.add(layers.Dropout(0.5)) # Dropout para evitar el overfitting

# Capa de salida
model.add(layers.Dense(9, activation='softmax'))
```

5.2. Compilación del modelo

```
In [39]: model.compile(
    loss='categorical_crossentropy', # Cambiado a 'categorical_crossentropy'
    optimizer='adam',
    metrics=['accuracy']
)
```

```
In [40]: model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Par
conv2d (Conv2D)	(None, 148, 148, 32)	
batch_normalization (BatchNormalization)	(None, 148, 148, 32)	
max_pooling2d (MaxPooling2D)	(None, 74, 74, 32)	
conv2d_1 (Conv2D)	(None, 72, 72, 64)	18
batch_normalization_1 (BatchNormalization)	(None, 72, 72, 64)	
max_pooling2d_1 (MaxPooling2D)	(None, 36, 36, 64)	
conv2d_2 (Conv2D)	(None, 34, 34, 128)	73
batch_normalization_2 (BatchNormalization)	(None, 34, 34, 128)	
max_pooling2d_2 (MaxPooling2D)	(None, 17, 17, 128)	
conv2d_3 (Conv2D)	(None, 15, 15, 128)	147
batch_normalization_3 (BatchNormalization)	(None, 15, 15, 128)	
max_pooling2d_3 (MaxPooling2D)	(None, 7, 7, 128)	
flatten (Flatten)	(None, 6272)	
dense (Dense)	(None, 512)	3,211
dropout (Dropout)	(None, 512)	
dense_1 (Dense)	(None, 9)	4

Total params: 3,458,633 (13.19 MB)

Trainable params: 3,457,929 (13.19 MB)

Non-trainable params: 704 (2.75 KB)

6. Ejecución del modelo

- Se entrenará el modelo utilizando Adam como optimizador y EarlyStopping para prevenir sobreajuste.
- Se evaluará el desempeño sobre el conjunto de prueba.

```
In [42]: early_stopping = EarlyStopping(
    monitor='val_loss', # Monitor the validation loss
    patience=5, # Number of epochs with no improvement after which training
    restore_best_weights=True # Restore the model weights from the epoch wi
)

from tensorflow.keras.callbacks import ReduceLR0nPlateau

reduce_lr = ReduceLR0nPlateau(
    monitor='val_loss',
    factor=0.2, # Reduce learning rate by a factor of 0.2
    patience=3, # Number of epochs with no improvement after which learning
    min_lr=1e-5 # Minimum learning rate
)
```

```
In [43]: history = model.fit(
    train_generator,
    epochs=EPOCHS,
    validation_data=validation_generator,
    callbacks=[early_stopping, reduce_lr],
    verbose=2
)
```

Epoch 1/8

4729/4729 - 594s - 126ms/step - accuracy: 0.6984 - loss: 1.1265 - val_accuracy: 0.8132 - val_loss: 0.5473 - learning_rate: 0.0010

Epoch 2/8

4729/4729 - 691s - 146ms/step - accuracy: 0.7788 - loss: 0.6960 - val_accuracy: 0.7562 - val_loss: 1.0302 - learning_rate: 0.0010

Epoch 3/8

4729/4729 - 1861s - 393ms/step - accuracy: 0.8194 - loss: 0.5604 - val_accuracy: 0.8512 - val_loss: 0.4386 - learning_rate: 0.0010

Epoch 4/8

4729/4729 - 626s - 132ms/step - accuracy: 0.8584 - loss: 0.4481 - val_accuracy: 0.9045 - val_loss: 0.2948 - learning_rate: 0.0010

Epoch 5/8

4729/4729 - 633s - 134ms/step - accuracy: 0.8845 - loss: 0.3619 - val_accuracy: 0.8376 - val_loss: 0.6183 - learning_rate: 0.0010

Epoch 6/8

4729/4729 - 638s - 135ms/step - accuracy: 0.9033 - loss: 0.3060 - val_accuracy: 0.9330 - val_loss: 0.2139 - learning_rate: 0.0010

Epoch 7/8

4729/4729 - 680s - 144ms/step - accuracy: 0.9159 - loss: 0.2675 - val_accuracy: 0.7954 - val_loss: 1.3952 - learning_rate: 0.0010

Epoch 8/8

4729/4729 - 715s - 151ms/step - accuracy: 0.9231 - loss: 0.2444 - val_accuracy: 0.9048 - val_loss: 0.4317 - learning_rate: 0.0010

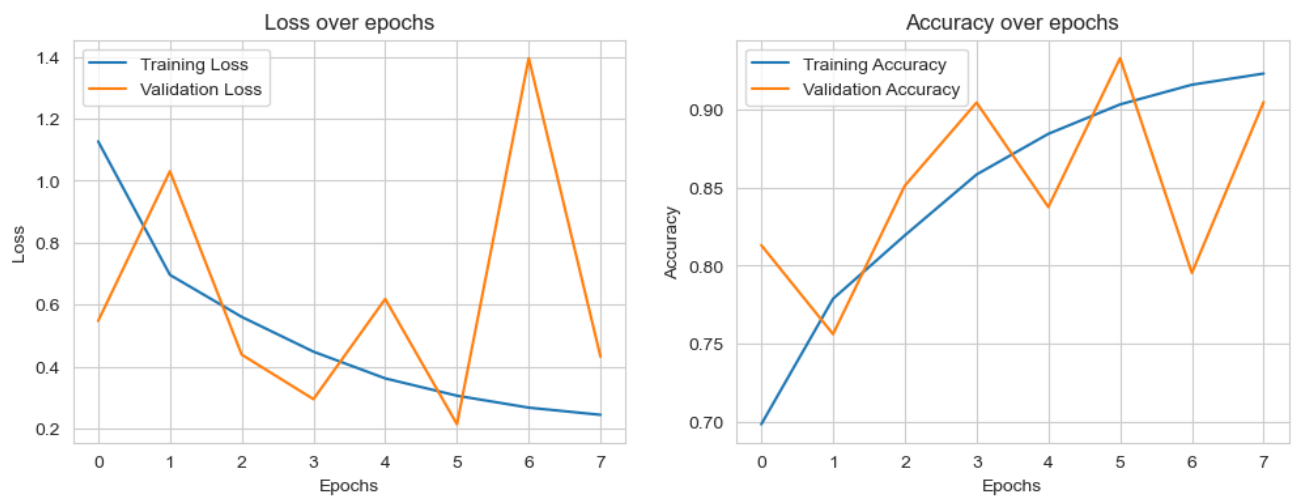
6.1. Resultados del Modelo

```
In [45]: # Acceder a los datos de pérdida y precisión
loss = history.history['loss']
val_loss = history.history['val_loss']
accuracy = history.history['accuracy']
val_accuracy = history.history['val_accuracy']

# Graficar la pérdida del modelo
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Loss over epochs')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()

# Graficar la precisión del modelo
plt.subplot(1, 2, 2)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Accuracy over epochs')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
```

Out[45]: <matplotlib.legend.Legend at 0x28ff7f1a0>



```
In [46]: epochs = [1, 2, 3, 4, 5, 6, 7, 8]
data = {
    'Epoch': epochs,
    'Training Loss': loss,
    'Validation Loss': val_loss,
    'Training Accuracy': accuracy,
    'Validation Accuracy': val_accuracy
}
```



```
}
df = pd.DataFrame(data)

# Imprimir la tabla
print(df)
```

	Epoch	Trainning Loss	Validation Loss	Training Accuracy \
0	1	1.126550	0.547342	0.698406
1	2	0.695954	1.030244	0.778794
2	3	0.560366	0.438560	0.819424
3	4	0.448124	0.294832	0.858415
4	5	0.361880	0.618251	0.884480
5	6	0.306024	0.213873	0.903328
6	7	0.267475	1.395219	0.915937
7	8	0.244362	0.431681	0.923101

	Validation Accuracy
0	0.813154
1	0.756159
2	0.851221
3	0.904515
4	0.837581
5	0.932960
6	0.795390
7	0.904832

```
In [130... # Accuracy final en entrenamiento
val_loss = history.history['val_loss'][-1]
```

```
In [140... print('Perdida final en Train:', history.history['loss'][-1])
print("Pérdida final en Validation:", history.history['val_loss'][-1])

print("Accuracy final en Train:", history.history['accuracy'][-1])
print("Accuracy final en Validation:", history.history['val_accuracy'][-1])
```

```
Perdida final en Train: 0.2443615049123764
Pérdida final en Validation: 0.43168148398399353
Accuracy final en Train: 0.9231013059616089
Accuracy final en Validation: 0.9048324227333069
```

```
In [ ]:
```

7. Evaluación del Modelo

7.1. Lote de imagenes de testeo

```
In [48]: from sklearn.metrics import confusion_matrix, accuracy_score, ConfusionMatrix
```



```
In [47]: fin = datetime.datetime.now()
print(f"Fin: {fin.strftime('%Y-%m-%d %H:%M:%S.%f')}")

duracion = fin - inicio
print(f"Duración: {duracion}")
```

Fin: 2025-04-04 15:22:39.664169

Duración: 1:47:37.571856

```
In [49]: test.Planta = test.Planta.astype('str')

test_generator = train_datagen.flow_from_dataframe(test,
                                                    directory = "Datos Finales",
                                                    subset = "training",
                                                    x_col = "image_id",
                                                    y_col = "Planta",
                                                    target_size = (TARGET_SIZE, TARGET_SIZE),
                                                    batch_size = BATCH_SIZE,
                                                    class_mode = "categorical")
```

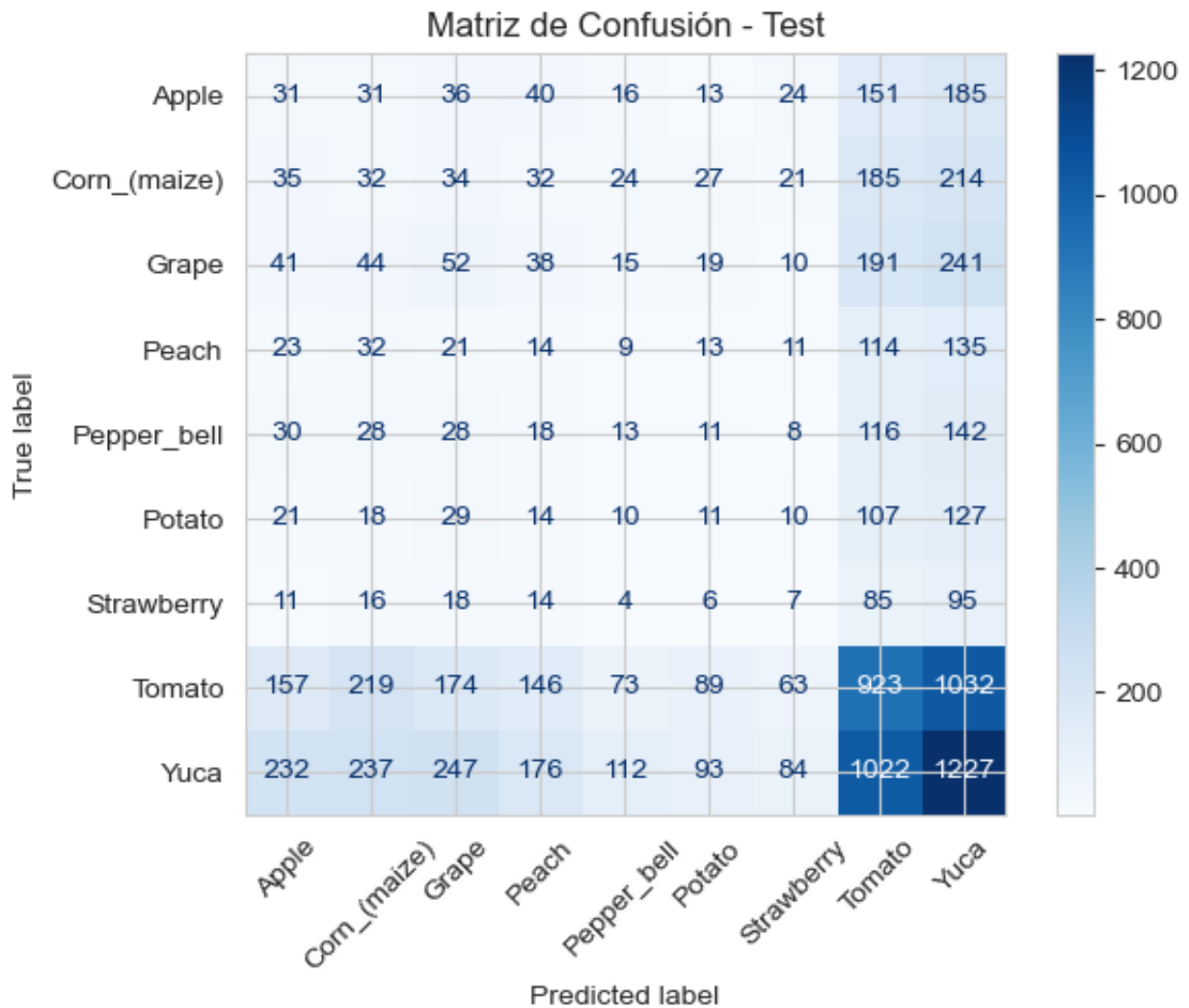
Found 9457 validated image filenames belonging to 9 classes.

7.2. Predicciones con imagenes de Testeo

```
In [104... # -----
# 1. Obtener predicciones completas para test
# -----
y_pred_probs = model.predict(test_generator)
y_pred = np.argmax(y_pred_probs, axis=1)
y_true = test_generator.classes
```

1183/1183  47s 39ms/step

```
In [105... # -----
# 2. Matriz de Confusión
# -----
cm = confusion_matrix(y_true, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=test_gener
disp.plot(cmap="Blues", xticks_rotation=45)
plt.title("Matriz de Confusión - Test")
plt.show()
```



In []:

7.3. Presición de la Predicción

```
In [106... accuracy = accuracy_score(y_true, y_pred)
print(f"Test Accuracy: {accuracy:.4f}")
```

Test Accuracy: 0.2443

7.4. Muestreo visual de predicciones vs etiquetas reales

```
In [162... # Función para normalizar imágenes solo para visualización
def normalize_for_display(img):
    img_min = img.min()
    img_max = img.max()
    if img_max > 1.0 or img_min < 0.0:
        img = (img - img_min) / (img_max - img_min + 1e-7)
```

```

    return img

# Obtener nombres de clases
class_indices = test_generator.class_indices
class_names = list(class_indices.keys())

# Reiniciar el generador para empezar desde el principio
test_generator.reset()
x_batch, y_true_batch = next(test_generator)

# Obtener predicciones del modelo
y_pred_probs_batch = model.predict(x_batch)
y_pred_batch = np.argmax(y_pred_probs_batch, axis=1)
y_true_batch = np.argmax(y_true_batch, axis=1)

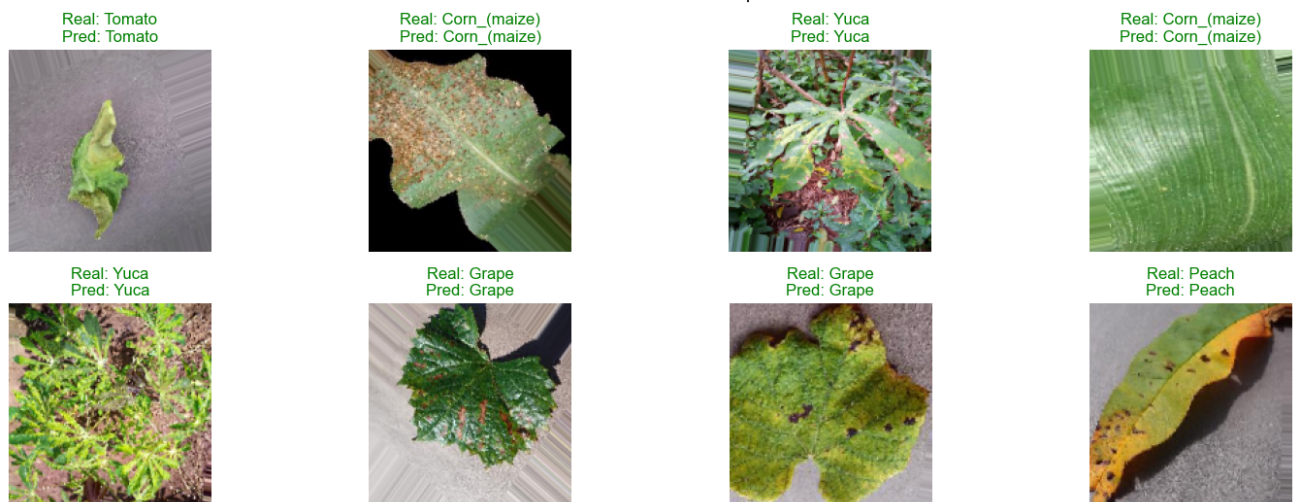
# Número de imágenes a mostrar (máximo el tamaño del batch)
num_samples = x_batch.shape[0]

# Mostrar imágenes con etiquetas
plt.figure(figsize=(15, 8))
for i in range(num_samples):
    plt.subplot(3, 4, i + 1)
    img_to_show = normalize_for_display(x_batch[i])
    plt.imshow(img_to_show)
    plt.axis('off')
    plt.title(
        f"Real: {class_names[y_true_batch[i]]}\nPred: {class_names[y_pred_batch[i]]}"
        color='green' if y_true_batch[i] == y_pred_batch[i] else 'red'
    )
plt.tight_layout()
plt.suptitle("Muestreo de Predicciones vs. Etiquetas Reales", fontsize=16, y
plt.show()

```

1/1 ————— 0s 124ms/step

Muestreo de Predicciones vs. Etiquetas Reales



In [164...

```

# Función para normalizar imágenes solo para visualización
def normalize_for_display(img):
    img_min = img.min()
    img_max = img.max()
    if img_max > 1.0 or img_min < 0.0:
        img = (img - img_min) / (img_max - img_min + 1e-7)
    return img

# Obtener nombres de clases
class_indices = test_generator.class_indices
class_names = list(class_indices.keys())

# Reiniciar el generador para empezar desde el principio
test_generator.reset()
x_batch, y_true_batch = next(test_generator)

# Obtener predicciones del modelo
y_pred_probs_batch = model.predict(x_batch)
y_pred_batch = np.argmax(y_pred_probs_batch, axis=1)
y_true_batch = np.argmax(y_true_batch, axis=1)

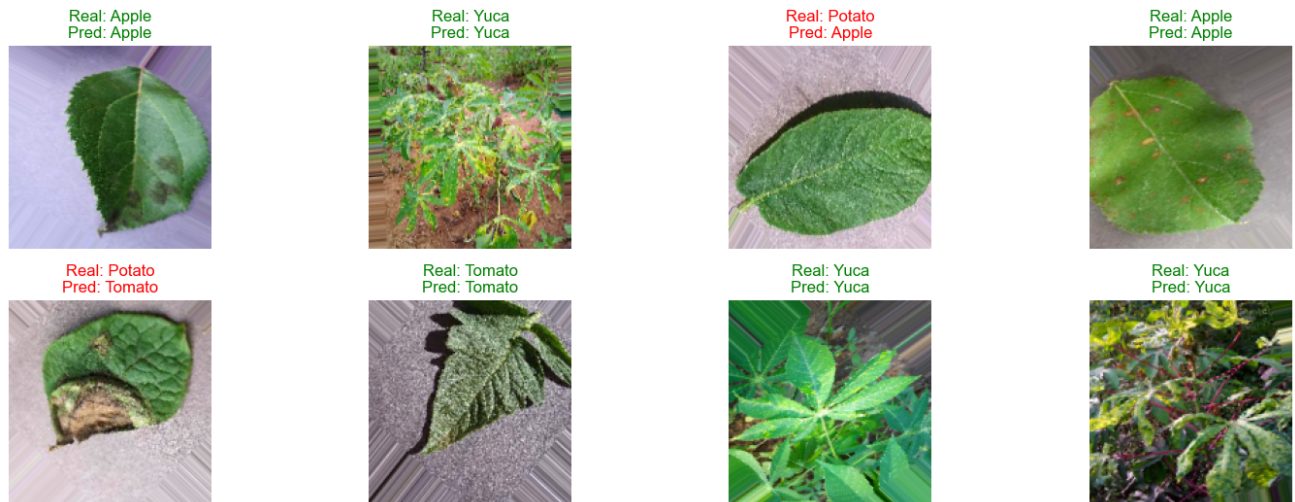
# Número de imágenes a mostrar (máximo el tamaño del batch)
num_samples = x_batch.shape[0]

# Mostrar imágenes con etiquetas
plt.figure(figsize=(15, 8))
for i in range(num_samples):
    plt.subplot(3, 4, i + 1)
    img_to_show = normalize_for_display(x_batch[i])
    plt.imshow(img_to_show)
    plt.axis('off')
    plt.title(
        f"Real: {class_names[y_true_batch[i]]}\nPred: {class_names[y_pred_batch[i]]}"
        color='green' if y_true_batch[i] == y_pred_batch[i] else 'red'
    )
plt.tight_layout()
plt.suptitle("Muestreo de Predicciones vs. Etiquetas Reales", fontsize=16, y
plt.show()

```

1/1 ————— 0s 67ms/step

Muestreo de Predicciones vs. Etiquetas Reales



In [172]...

```

from sklearn.metrics import roc_curve, auc
from tensorflow.keras.utils import to_categorical

# 3. Convertir etiquetas reales a one-hot
n_classes = y_pred_probs.shape[1]
y_true_onehot = to_categorical(y_true, num_classes=n_classes)

# 4. Calcular ROC y AUC por clase
fpr = dict()
tpr = dict()
roc_auc = dict()

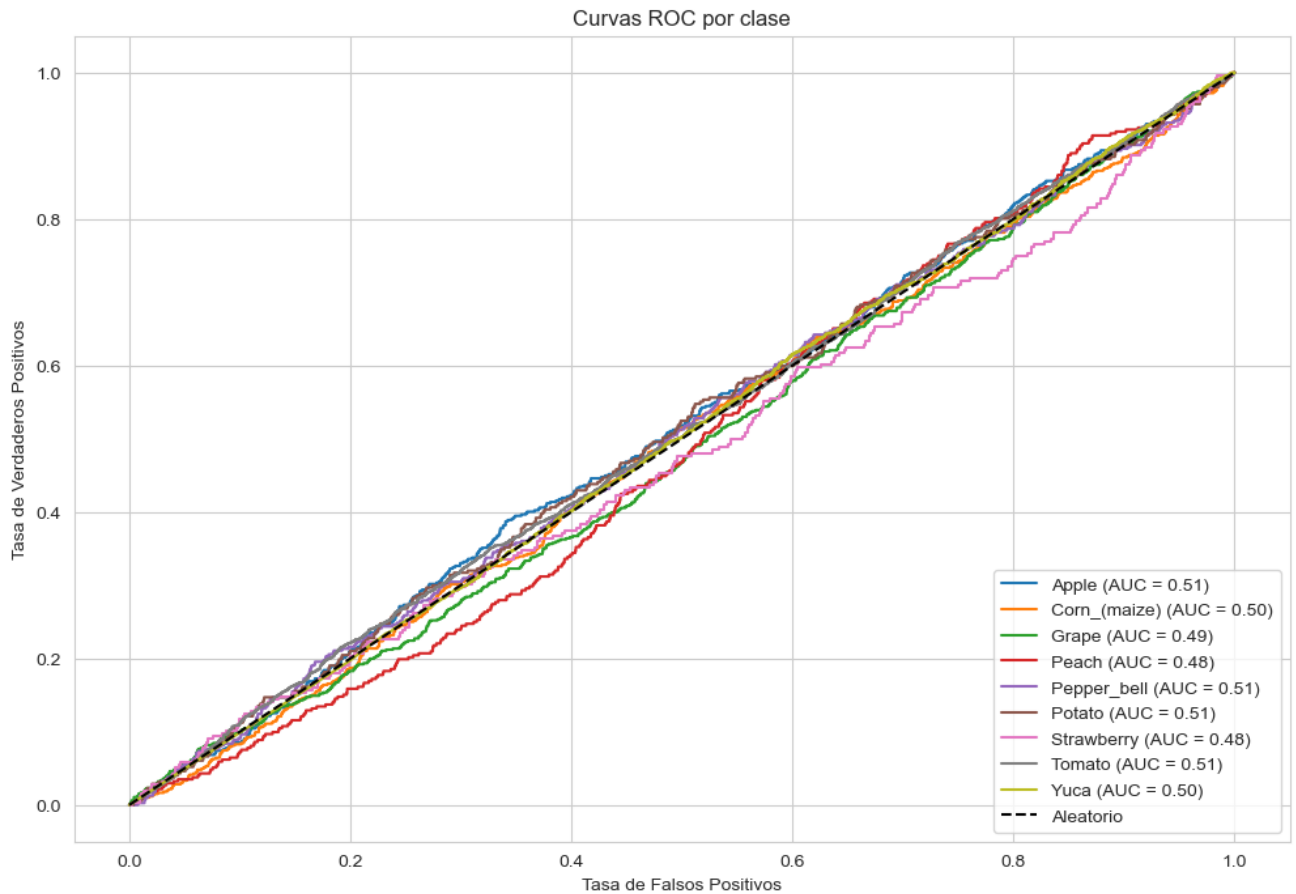
for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_true_onehot[:, i], y_pred_probs[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

# 5. Obtener nombres de clase
class_names = list(test_generator.class_indices.keys())

# 6. Graficar curvas ROC
plt.figure(figsize=(12, 8))
for i in range(n_classes):
    plt.plot(fpr[i], tpr[i], label=f'{class_names[i]} (AUC = {roc_auc[i]:.2f})')

plt.plot([0, 1], [0, 1], 'k--', label='Aleatorio')
plt.xlabel('Tasa de Falsos Positivos')
plt.ylabel('Tasa de Verdaderos Positivos')
plt.title('Curvas ROC por clase')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()

```



```
In [175... from sklearn.metrics import roc_auc_score

# AUC macro (promedio de clases, todas igual de importantes)
auc_macro = roc_auc_score(y_true_onehot, y_pred_probs, average='macro')

# AUC micro (promedia sobre todas las instancias)
auc_micro = roc_auc_score(y_true_onehot, y_pred_probs, average='micro')

print(f"AUC Macro promedio: {auc_macro:.4f}")
print(f"AUC Micro promedio: {auc_micro:.4f}")
```

AUC Macro promedio: 0.4980

AUC Micro promedio: 0.6066